

CLASE 2 — Introducción a JavaScript para Backend

Objetivos de la Clase

Al finalizar la clase el estudiante será capaz de:

- Comprender los fundamentos de JavaScript usados en backend
 - Declarar variables correctamente
 - Crear funciones reutilizables
 - Utilizar arrow functions
 - Manipular arrays y objetos
 - Aplicar métodos funcionales:
 - `map()`
 - `filter()`
 - `find()`
 - Resolver problemas usando estructuras de datos
 - Preparar la lógica para APIs y bases de datos
-

Duración Estimada

Bloque	Tiempo
Introducción	20 min
Variables y Tipos de Datos	35 min
Funciones y Arrow Functions	45 min
Arrays y Objetos	40 min
<code>map()</code> , <code>filter()</code> , <code>find()</code>	50 min
Taller práctico	50 min
Preguntas y cierre	15 min

Introducción

¿Por qué aprender JavaScript para backend?

JavaScript ya no solo funciona en navegadores.

Gracias a Node.js ahora podemos crear:

- APIs
- servidores
- sistemas backend
- microservicios
- aplicaciones en tiempo real

Todo usando JavaScript.

¿Qué es Node.js?

Node.js es un entorno de ejecución que permite correr JavaScript fuera del navegador.

Ejemplo:

```
node app.js
```

VARIABLES

¿Qué es una variable?

Una variable es un espacio donde almacenamos información.

let

Permite crear variables que pueden cambiar.

```
let nombre = "Juan";
```

```
nombre = "Carlos";
```

```
console.log(nombre);
```

Resultado:

Carlos

const

Se usa cuando el valor no debe cambiar.

```
const pi = 3.1416;
```

```
console.log(pi);
```

var (NO recomendado)

Forma antigua de declarar variables.

```
var edad = 20;
```

Problemas:

- alcance confuso
- errores difíciles de detectar

Hoy se usa:

- let
 - const
-

Tipos de Datos

String

```
const nombre = "Ana";
```

Number

```
const edad = 25;
```

Boolean

```
const activo = true;
```

Array

```
const numeros = [1,2,3];
```

Object

```
const usuario = {  
  nombre: "Juan",  
  edad: 20  
};
```

Ejercicio 1

Crear variables para:

- nombre de estudiante
- edad
- ciudad
- si está activo

Mostrar todo con console.log()

FUNCIONES

¿Qué es una función?

Bloque reutilizable de código.

Función tradicional

```
function saludar() {  
  console.log("Hola");
```

```
}
```

```
saludar();
```

Función con parámetros

```
function saludar(nombre) {  
  console.log("Hola " + nombre);  
}
```

```
saludar("Juan");
```

Función con retorno

```
function sumar(a, b) {  
  return a + b;  
}
```

```
const resultado = sumar(10, 5);
```

```
console.log(resultado);
```

Ejercicio 2

Crear funciones para:

- sumar
 - restar
 - multiplicar
 - dividir
-

ARROW FUNCTIONS

Sintaxis moderna

```
const saludar = () => {  
  console.log("Hola");  
}
```

```
};
```

Arrow function con parámetros

```
const sumar = (a, b) => {  
  return a + b;  
};
```

Retorno implícito

```
const multiplicar = (a, b) => a * b;
```

Diferencias principales

Tradicional	Arrow
function()	() =>
Más extensa	Más moderna
Usa this dinámico	this léxico

Ejercicio 3

Convertir funciones tradicionales a arrow functions.

ARRAYS

¿Qué es un array?

Colección de elementos.

```
const frutas = ["manzana", "pera", "uva"];
```

Acceder a elementos

```
console.log(frutas[0]);
```

Recorrer arrays

```
for (let fruta of frutas) {  
  console.log(fruta);  
}
```

push()

Agregar elementos.

```
frutas.push("sandia");
```

length

```
console.log(frutas.length);
```

OBJETOS

¿Qué es un objeto?

Estructura clave-valor.

```
const usuario = {  
  nombre: "Ana",  
  edad: 22,  
  activo: true  
};
```

Acceder a propiedades

```
console.log(usuario.nombre);
```

Modificar propiedades

usuario.edad = 30;

Arrays de objetos

```
const usuarios = [  
  {  
    nombre: "Juan",  
    edad: 20  
  },  
  {  
    nombre: "Ana",  
    edad: 17  
  }  
];
```

MÉTODOS IMPORTANTES

map()

Transforma elementos.

```
const numeros = [1,2,3];
```

```
const dobles = numeros.map(n => n * 2);
```

```
console.log(dobles);
```

Resultado:

```
[2,4,6]
```

filter()

Filtra elementos.


```
const usuarios = [
  {nombre:"Juan", edad:20},
  {nombre:"Ana", edad:17}
];

const mayores = usuarios.filter(
  u => u.edad >= 18
);

console.log(mayores);
```

find()

Busca el primer elemento.

```
const usuario = usuarios.find(
  u => u.nombre === "Ana"
);

console.log(usuario);
```

Diferencia entre filter y find

filter	find
Retorna array	Retorna objeto
Puede traer varios	Solo uno

Ejercicio Guiado

Sistema de Productos

```
const productos = [
  {
    id: 1,
    nombre: "Laptop",
    precio: 3000
  }
];
```

```
},  
{  
  id: 2,  
  nombre: "Mouse",  
  precio: 80  
},  
{  
  id: 3,  
  nombre: "Teclado",  
  precio: 150  
}  
];
```

Obtener nombres con map

```
const nombres = productos.map(  
  p => p.nombre  
);
```

```
console.log(nombres);
```

Filtrar productos caros

```
const caros = productos.filter(  
  p => p.precio > 100  
);
```

```
console.log(caros);
```

Buscar producto

```
const producto = productos.find(  
  p => p.id === 2  
);
```

```
console.log(producto);
```

TALLER PRÁCTICO

Parte 1 — Usuarios

Crear un array de usuarios:

```
const usuarios = [  
  {  
    id: 1,  
    nombre: "Juan",  
    edad: 20  
  },  
  {  
    id: 2,  
    nombre: "Ana",  
    edad: 17  
  },  
  {  
    id: 3,  
    nombre: "Pedro",  
    edad: 30  
  }  
];
```

Ejercicios

1. Filtrar mayores de edad

Crear función:

filtrarMayores()

2. Buscar usuario por nombre

Crear función:

buscarUsuario(nombre)

3. Obtener solo nombres

Usar map()

Parte 2 — Productos

```
const productos = [  
  {  
    id: 1,  
    nombre: "Laptop",  
    precio: 3000  
  },  
  {  
    id: 2,  
    nombre: "Mouse",  
    precio: 80  
  },  
  {  
    id: 3,  
    nombre: "Monitor",  
    precio: 1200  
  }  
];
```

Ejercicios

1. Filtrar productos costosos

Precio mayor a 500.

2. Buscar producto por ID

3. Calcular total

Usar reduce()

```
const total = productos.reduce(  
  (acumulador, producto) =>  
    acumulador + producto.precio,  
  0  
);
```

```
console.log(total);
```

RETO FINAL

Crear sistema simple de carrito.

Debe permitir:

- listar productos
 - buscar productos
 - filtrar productos
 - calcular total del carrito
-

Buenas Prácticas

Usar const por defecto

```
const nombre = "Juan";
```

Nombres descriptivos

```
const usuariosActivos = [];
```

NO:

```
const u = [];
```

Funciones pequeñas

Cada función debe hacer una sola tarea.

Errores comunes

Confundir = con ===

```
if (edad === 18)
```

Modificar arrays accidentalmente

No retornar valores

```
const sumar = (a,b) => {  
  a + b;  
}
```

Incorrecto.

Resumen

Hoy aprendimos:

- Variables
 - Tipos de datos
 - Funciones
 - Arrow functions
 - Arrays
 - Objetos
 - map()
 - filter()
 - find()
 - reduce()
-

Tarea

Crear un sistema de estudiantes que permita:

- agregar estudiantes
- buscar estudiantes
- filtrar aprobados
- calcular promedio general

Usando:

- arrays
- objetos

- funciones
- map/filter/find/reduce